Grammars and ASTs

CS-3180

Lisp

- Lisp is a very simple language (to parse)
- Complexity arises out of it
- We'll spend today talking about *how* languages are parsed
- Deeper knowledge of Lisp opens the door to more complex features



Language Parsing

- When an executor ingest a source
- Its fundamentally just a string parser
- How do you usually parse strings?

Language Parsing

- When an executor ingest a source
- Its fundamentally just a string parser
- How do you usually parse strings?
- Sub-strings?

Language Parsing

- When an executor ingest a source
- Its fundamentally just a string parser
- How do you usually parse strings?
- Sub-strings?
- Split?

Language Parsing

- When an executor ingest a source
- Its fundamentally just a string parser
- How do you usually parse strings?
- Sub-strings?
- Split?
- Regex!!!!!!

Regex

- Talk about regex in terms of "matches
- This string is "accepted"
- This string is "not accepted"

```
# \s: any lowercase
# \S: any lowercase
# +: 1 or more time (* for 0
[\s\S]+@wright.edu
```

```
reese.hatfield@wright.edu
```

```
# random-non-email-fail
```

```
smith.2@wright.edu
```

Regex

- You can parse a LOT of complex things with regex
- Infamous SO post
- But it has its limitations

https://stackoverflow.com/questions/201323/how-can-i-validate-an-email-address-using-a-regular-expression

Regex

- You can parse a LOT of complex things with regex
- Infamous SO post
- But it has its limitations
- It's ugly

https://stackoverflow.com/questions/201323/how-can-i-validate-an-email-address-using-a-regular-expression

Regex

- You can parse a LOT of complex things with regex
- Infamous SO post
- But it has its limitations
- It's ugly
- It's hard to maintain

https://stackoverflow.com/questions/201323/how-can-i-validate-an-email-address-using-a-regular-expression

Regex

- You can parse a LOT of complex things with regex
- Infamous SO post
- But it has its limitations
- It's ugly
- It's hard to maintain
- There are simple strings it CANNOT parse

https://stackoverflow.com/questions/201323/how-can-i-validate-an-email-address-using-a-regular-expression

Regex Failures

- Programming languages often have nested components
- `<` and `>` -> SGML languages
- `(` and `)` -> Lisp Languages
- `{` and `}` -> C-style languages
- Mix these, but they must ALWAYS match

Regex Parsing

- Consider a simplest possible example:
- Detect matching parentheses
- How would you do this with regex?
- Regex can't "remember" how many parentheses came before
- Limitation of state machines

```
((()())) # accept
```

```
(((() # fail
```

Better than Regex

- Syntax is more expressive than regular languages
- We need something more advanced than Regex
- (Preferably more than a PDA)
- Enter the (Context-Free) Grammar

Context Free Grammars (CFGs)

- A CFG is defined as a 4-tuple
- V: Variables (non-terminal placeholders)
- T: Terminals (concrete strings)
- P: Productions (replacement mappings from Variables to Variables/Terminals)
- S: Start (A beginning for strings to start deriving)

CFGs

- Easier to see an example of
- V = `{ S }`
- T = `EMPTY_STRING`
- P = `{ S -> (S)S | EMPTY_STRING }`
- S = `S`
- Start with S, replace S with production rules

CFG Derivations

- Using that grammar let's test some things
- P = `{ S -> (S)S | EMPTY_STRING }`
- `((((()))))`
- `((((()()`
- Proving this is much trickier
- Many formal definitions of this (we're skipping)
- Let's look at one more slightly bigger example

CFG Continued

- What "language" does this accept?
- V = `{ S, P }`
- T = `{0, 1}`
- P = `S -> 0P | EMPTY_STRING`
- `P -> S11`
- S = `S`

CFG Continued

- What "language" does this accept?
- V = `{ S, P }`
- T = `{0, 1}`
- P = `S -> 0P | EMPTY_STRING`
- `P -> S11`
- S = `S`
- Any amount of 0s, followed by twice that many 1s

Lisp's Grammar

- Very simple
- "Backus-Naur Form" is a grammar "notation" that makes this easy to see
- Mix of Regex (*, +, etc) with formal CFG

```
1. s-expr  ::= atom | list
2. list   ::= "(" s-expr* ")"
```

- Procedural languages have more complex grammars
- When you go on to parse Lisp, this will be useful

Parsing

- Once the rules are defined
- Source is "lexed" into tokens/symbols (!)
- Symbols are then parsed into a tree
- Production rules in the grammar
- Map directly to nodes in the tree
- Terminals map to leaf nodes

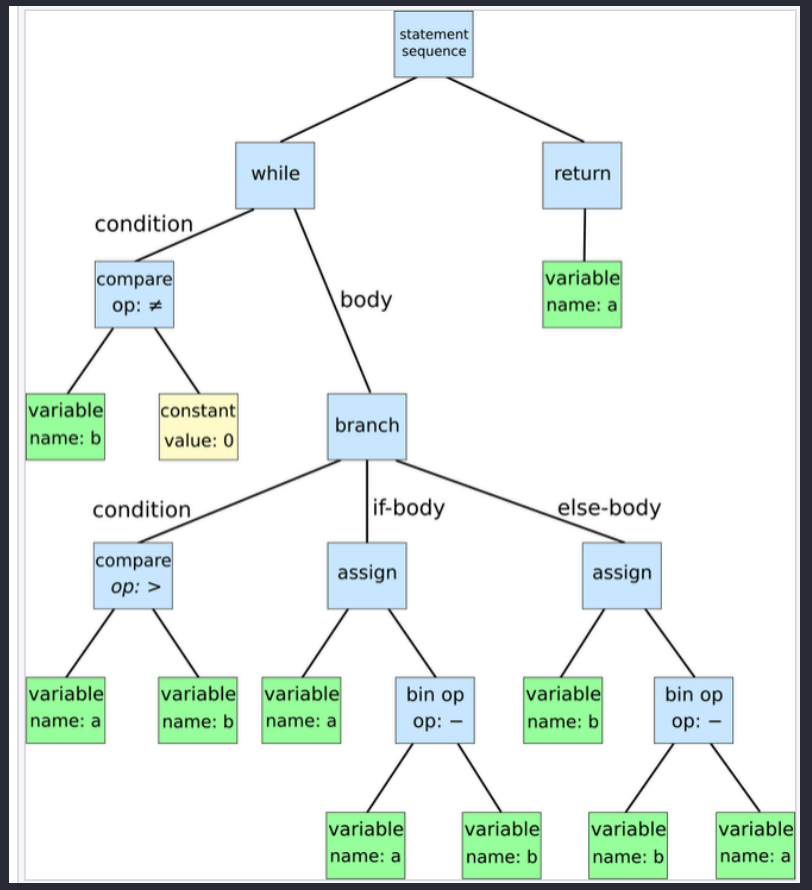
Syntax Trees

- Concrete Syntax Trees represent the raw source
- Usually converted to an Abstract Syntax Tree
- (dropping non-essential information like {, }, ;, etc)
- Optimizations usually occur at the AST level

AST example

- Euclidean Algorithm
- Straight from Wikipedia

```
1 while b != 0:
2     if a > b:
3         a := a - b
4     else:
5         b := b - a
6 return a
```



Execution

- Once you have a syntax tree, you can start parsing the code
- Walk the AST (pre-order or post-order)
- Simple AST walking is slow, but it IS how CommonLisp handles this
- Each Node is a `cons` cell
- You could compile into a lower representation
- Or attempt to optimize AST walking

Summary

- This is a generalized process
- Most languages tweaks a few ideas about how parsing/execution works
- ALL* of them at least need:

- "Symbols step" will be important for next time

Summary

- This is a generalized process
- Most languages tweaks a few ideas about how parsing/execution works
- ALL* of them at least need:
- Lexed into tokens/symbols

- "Symbols step" will be important for next time

Summary

- This is a generalized process
- Most languages tweaks a few ideas about how parsing/execution works
- ALL* of them at least need:
- Lexed into tokens/symbols
- Built into an AST

- "Symbols step" will be important for next time

Summary

- This is a generalized process
- Most languages tweaks a few ideas about how parsing/execution works
- ALL* of them at least need:
- Lexed into tokens/symbols
- Built into an AST
- Executed (at some level)
- "Symbols step" will be important for next time

Questions